



Chương 5: Thiết kế chương trình và phân tích

Computer Engineering Faculty

Embedded Systems

Nội Dung



1

Mô hình thiết kế cho ngôn ngữ cấp cao

2

Quy trình chuyển từ ngôn ngữ cấp cao thành mã lệnh

3

Kỹ thuật phân tích hiệu năng chương trình

4

Phương pháp tạo trường hợp test

5

Bài tập

Mô hình thiết kế



❖ Design Flow Graph

□ Một design flow graph là một mô hình không có điều kiện. Trong ngôn ngữ lập trình cấp cao, một đoạn code không có câu lệnh điều kiện, với chỉ một điểm vào và một điểm thoát được gọi là một **khối cơ bản**.

□ Ví dụ một khối cơ bản trong C:

$$W = a + b;$$

$$X = a - c;$$

$$Y = x + d;$$

$$X = a + c;$$

$$Z = y + e;$$

Mô hình thiết kế



❖ Design Flow Graph (DFG)

- ❑ Trước khi vẽ DFG, ta phải tiến hành biến đổi code sao cho đảm bảo tính thực hiện tuần tự của các dòng code ở dạng những câu lệnh gán đơn, trong đó một biến chỉ xuất hiện một lần ở bên trái
- ❑ Ví dụ biến đổi sang dạng câu lệnh gán đơn:

$W = a + b;$

$X = a - c;$

$Y = x + d;$

$X = a + c;$

$Z = y + e;$



$W = a + b;$

$X1 = a - c;$

$Y = x1 + d;$

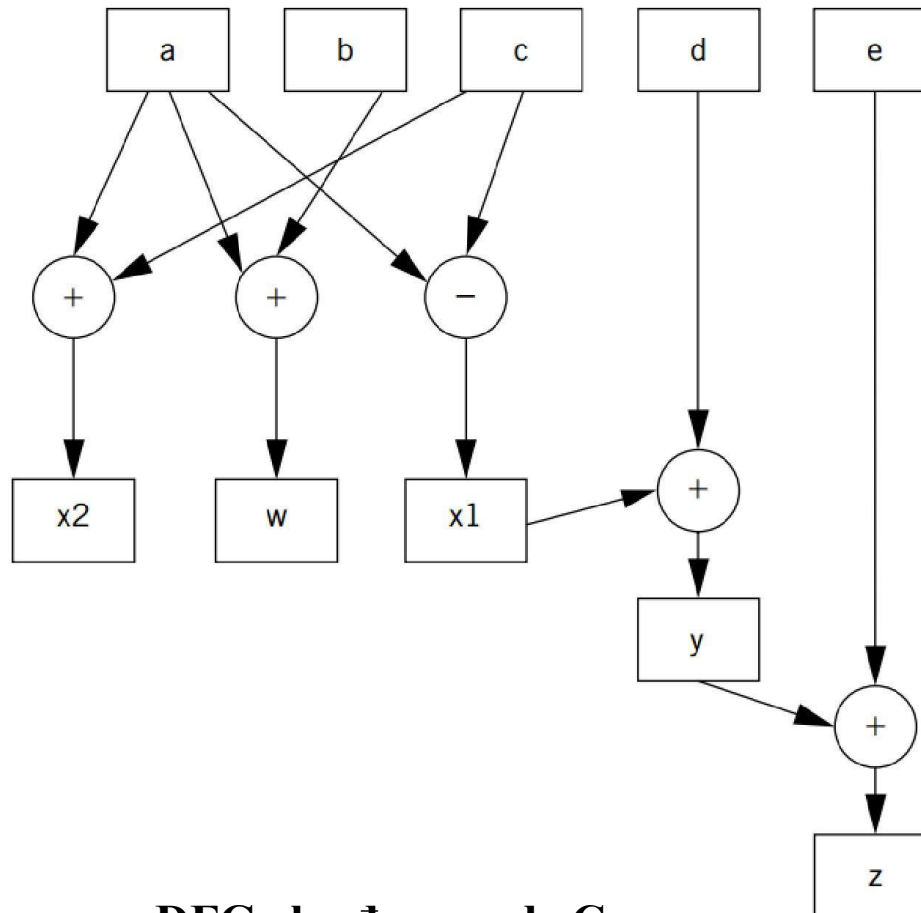
$X2 = a + c;$

$Z = y + e;$

Design Flow Graph



□ Vẽ DFG



DFG cho đoạn code C

Design Flow Graph



- ❑ DFG tạo ra thứ tự thực hiện các câu lệnh trở nên rõ ràng hơn các câu lệnh thực hiện trên C
- ❑ Sử dụng DFG chúng ta có thể thực hiện việc sắp xếp lại các câu lệnh sao cho giảm tải cho pipeline hoặc xung đột cache.

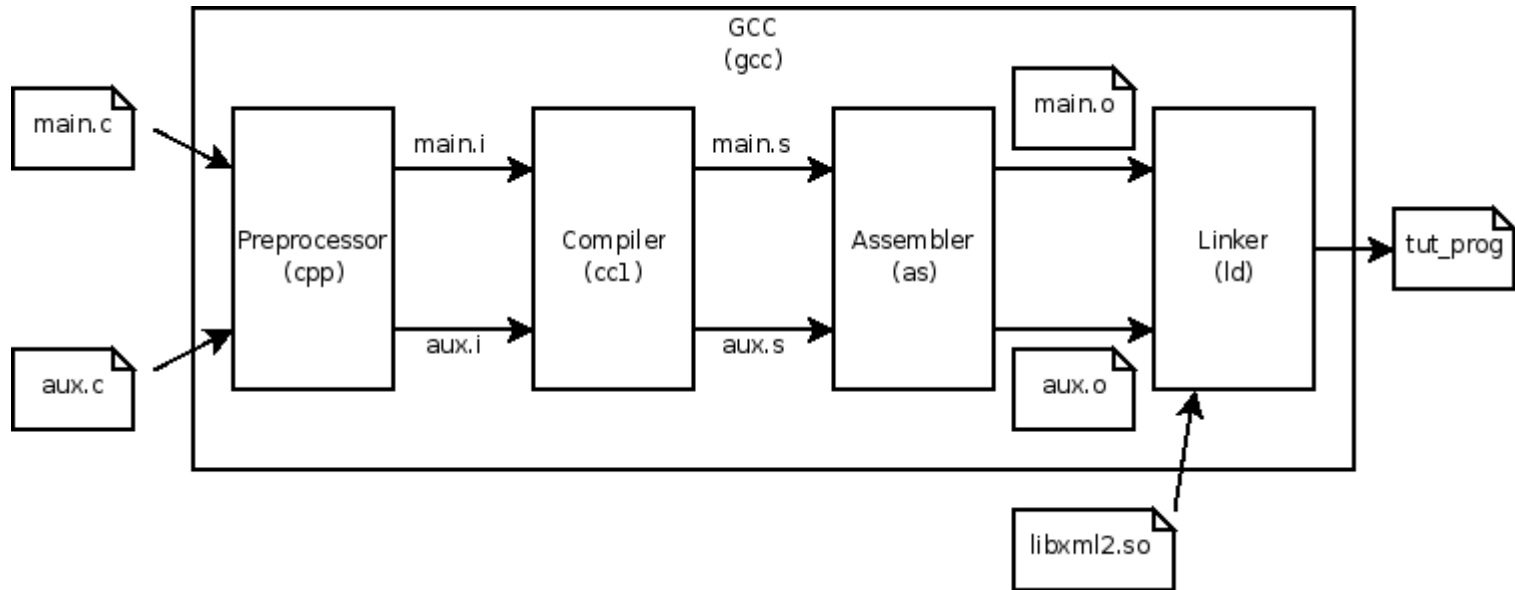
DFG cho đoạn code C

Quy trình chuyển từ ngôn ngữ cấp cao thành mã lệnh



- ❑ Quá trình chuyển đổi từ ngôn ngữ cấp cao thành mã lệnh được sự tham gia của một chương trình gọi là compiler (chương trình dịch)
- ❑ Quy trình dịch bao gồm
 - Giai đoạn tiền xử lý:
 - Phân tích từ vựng(Lexical Analysis), phân tích cú pháp (Syntax Analysis), phân tích ngữ nghĩa
 - Tiền xử lý(preprocessing): nhận mã nguồn và xoá bỏ các chú thích
 - Giai đoạn dịch: sinh mã
 - Giai đoạn liên kết: mã máy của một chương trình dịch từ nhiều nguồn khác nhau được liên kết lại với nhau để tạo thành chương trình đích duy nhất.

Quy trình chuyển từ ngôn ngữ cấp cao thành mã lệnh



Sơ đồ quá trình dịch ngôn ngữ C

Quy trình chuyển từ ngôn ngữ cấp cao thành mã lệnh



□ Ví dụ:

Temp = v[k];

v[k] = v[k+1];

v[k+1] = temp;

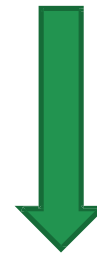


lw \$15, 0(\$2)

lw \$16, 4(\$2)

lw \$16, 0(\$2)

lw \$15, 4(\$2)



```
0000 1001 1100 0110 1010 1111 0101 1000
1010 1111 0101 1000 0000 1001 1100 0110
1100 0110 1010 1111 0101 1000 0000 1001
0101 1000 0000 1001 1100 0110 1010 1111
```

Phân tích hiệu năng chương trình



□ Tổng quan:

- Để đánh giá hiệu năng chương trình, ta tiến hành đo thời gian thực thi nhiệm vụ của chương trình đó.

□ Hướng tiếp cận

- Đo đạc thời gian thực thi trong trường hợp tệ nhất cũng như các trường hợp khác tương ứng với các nhánh thực thi khác nhau của chương trình
- Để phép đo chính xác cho từng loại CPU, ta cần thực hiện trên chính CPU đó hoặc công cụ mô phỏng



Phân tích hiệu năng chương trình



□ Program trace

- Tiến hành lưu lại những nhánh thực thi của chương trình như là program trace.
- Dữ liệu được ghi lại đó cũng là cơ sở để phân tích hành vi của chương trình

□ Để đo đạc các nhánh thực thi khác nhau đòi hỏi người đo phải định nghĩa được một tập các dữ liệu đầu vào tương ứng, bao phủ toàn bộ các nhánh thực thi đó.

□ Đây là vấn đề khó

Phương pháp tạo tập test



□ Một số phương pháp đề nghị

- Sử dụng Timer nếu có thể: bật timer khi tiến hành đo hiệu năng một hàm hoặc module nào đó, kích hoạt, đưa input thích hợp và ghi nhận kết quả.
- Kết hợp với logic analyser hoặc oscilloscope để đo các tín hiệu đánh dấu việc kết thúc chương trình/module. Tuy nhiên hướng này không thích hợp với chương trình có thời gian thực thi lâu
 - Tại sao?

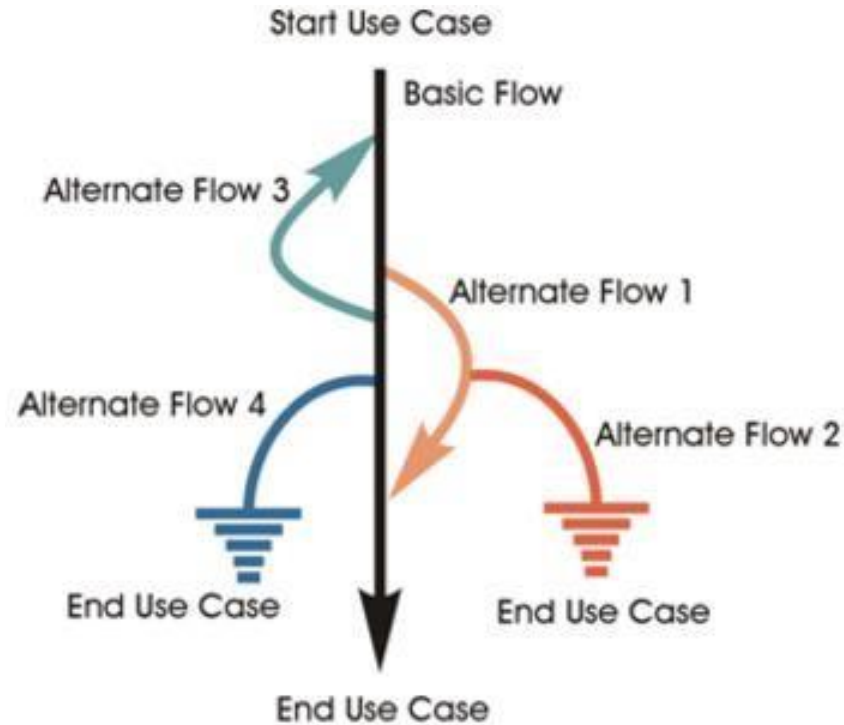
Phương pháp tạo tập test



□ Sự cần thiết phải có use case:

- Use Case là một công cụ xuất sắc để khuyến khích những người dùng tiềm năng nói về hệ thống từ hướng nhìn của họ
- Use case tạo nên một lời mô tả rõ ràng và nhất quán về việc hệ thống cần phải làm gì
- Có thể đơn giản hóa việc thay đổi và mở rộng hệ thống qua việc thay đổi và mở rộng mô hình Use Case, sau đó chỉ theo dõi riêng những Use Case đã bị thay đổi cùng những hiệu ứng của chúng trong thiết kế hệ thống và xây dựng hệ thống

Phương pháp tạo tập test



Mô hình luồng các sự kiện trong một use case

Phương pháp tạo tập test



□ Lập test cases từ use case

- Các bước lập test case từ use case
- Tìm ra các “path” của các luồng sự kiện trong use case. Mỗi path như này được gọi là một scenario
- Chỉ ra điều kiện xảy ra mỗi scenario
- Mô tả điều kiện, trình tự diễn ra trong mỗi scenario và kết quả mong muốn.

Phương pháp tạo tập test



☐ Theo dõi và quản lý các test case

Các bước chuẩn bị test

- Phân tích requirements
- Lập các kế hoạch test
- Lập testing checklist
- Mô tả các test case phức tạp
- Theo dõi các thay đổi
- Phân tích dữ liệu giả định
- Lập test procedure cho nhiều test case
- Sử dụng Excel



UIT-HCM



Thank You !